Quant Research Template Guide v3

Poetry + Docker + VS Code Dev Containers + Jupyter for Quant Research

1. Goal

Build a portable research environment where VS Code connects to a Docker container, Jupyter notebooks execute inside the container, and no host Python installation is required. Clone the repository on any future machine and get the exact same environment.

2. Architecture

Mental model:

```
Git Repo
    ↓
Dockerfile
    ↓
Docker Image
    ↓
Docker Container
    ↓
VS Code Dev Container
    ↓
Jupyter Notebook Kernel
```

3. Create the Project

Start with a standard Poetry project.

```
mkdir ~/quant_research
cd ~/quant_research

poetry new quant-template
cd quant-template
```

4. Add Research Directories

Create the folders commonly used in quant research.

```
mkdir notebooks data models scripts
```

5. Install Dependencies

Install the initial research stack.

```
poetry add numpy pandas matplotlib scipy jupyterlab ipykernel

poetry install
```

6. Dockerfile

The Dockerfile defines how the Docker image is built.

```
FROM python:3.14-slim

WORKDIR /workspace

RUN pip install poetry

COPY pyproject.toml poetry.lock ./

RUN poetry config virtualenvs.create false

RUN poetry install --no-root

COPY . .

RUN python -m ipykernel install     --user     --name quant-template     --display-name "Python (Quan

CMD ["bash"]
```

Dockerfile Explained

FROM: base image used to build the environment. WORKDIR: default directory inside the container. RUN: executes commands during image creation. COPY: copies files from the repository into the image. virtualenvs.create false: installs dependencies directly into the container Python. ipykernel install: registers a Jupyter kernel tied to the container's Python interpreter. CMD: default command executed when the container starts.

7. docker-compose.yml

Docker Compose defines how the container should run.

```
services:
  quant:
    build:
      context: .

    container_name: quant-template

    volumes:
      - .:/workspace

    working_dir: /workspace

    tty: true
```

docker-compose Explained

build.context: build image from the current directory. container_name: friendly name shown in Docker Desktop. volumes: mounts the repository into the container. Changes appear immediately. working_dir: default location when opening a shell. tty: keeps an interactive terminal available.

8. VS Code Dev Container

Create .devcontainer/devcontainer.json

```
{
  "name": "Quant Template",
  "dockerComposeFile": "../docker-compose.yml",
  "service": "quant",
  "workspaceFolder": "/workspace",
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-toolsai.jupyter"
      ]
    }
  }
}
```

9. Open in Container

Open the repository in VS Code and attach to Docker.

```
Command Palette
→ Dev Containers: Reopen in Container
```

10. Verify Python and Jupyter

Confirm that notebooks use Python from inside Docker.

```
which python

Expected:

/usr/local/bin/python
```

```
import sys
print(sys.executable)

Expected:

/usr/local/bin/python
```

11. Understanding the Jupyter Kernel Registration

The following command creates a named Jupyter kernel.

```
RUN python -m ipykernel install     --user     --name quant-template     --display-name "Python (Quan
```

What It Does

Creates a kernelspec, registers it with Jupyter, and points it to the container's Python interpreter. This helps VS Code discover and use the correct environment.

```
jupyter kernelspec list

cat /root/.local/share/jupyter/kernels/quant-template/kernel.json
```

12. Rebuilding, Restarting and Managing Containers

This is one of the most important Docker concepts.

When You MUST Rebuild

- Dockerfile changes - pyproject.toml changes - docker-compose.yml changes - Python dependency changes

When You DO NOT Need to Rebuild

- Notebook changes - Source code changes - README changes Because the repository is mounted live into the container.

```
docker compose build

docker compose up -d --build

docker compose down

docker compose up -d

VS Code:
Dev Containers → Rebuild Container
```

Recommended Workflow

For most development, simply edit code and notebooks. Only rebuild when the environment changes. The VS Code 'Rebuild Container' command is usually the easiest option.

13. Docker Desktop

After opening the project you should see: • One Docker Image • One Docker Container Both are visible and manageable through Docker Desktop.

14. Create Future Projects From This Template

Once the repository is working, convert it into a GitHub template.

```
git init
```

```
git add .
git commit -m "Initial template"

gh repo create quant-template   --public   --source=.   --push
```

Enable Template Repository

GitHub Repository → Settings → Template Repository

```
gh repo create my-new-project   --template yourusername/quant-template
```

What New Projects Inherit

Every future project automatically receives: • Dockerfile • docker-compose.yml • .devcontainer • Poetry configuration • Jupyter configuration • Research folder structure